Semantic tagging

T3000 MCP Server — Design · built Nov 13, 2026

T3000 MCP Server — Design

> **Current state**: 50+ tools across 16 categories (v5 expanded from v4). This doc describes the v4 baseline (44 tools, 11 categories).

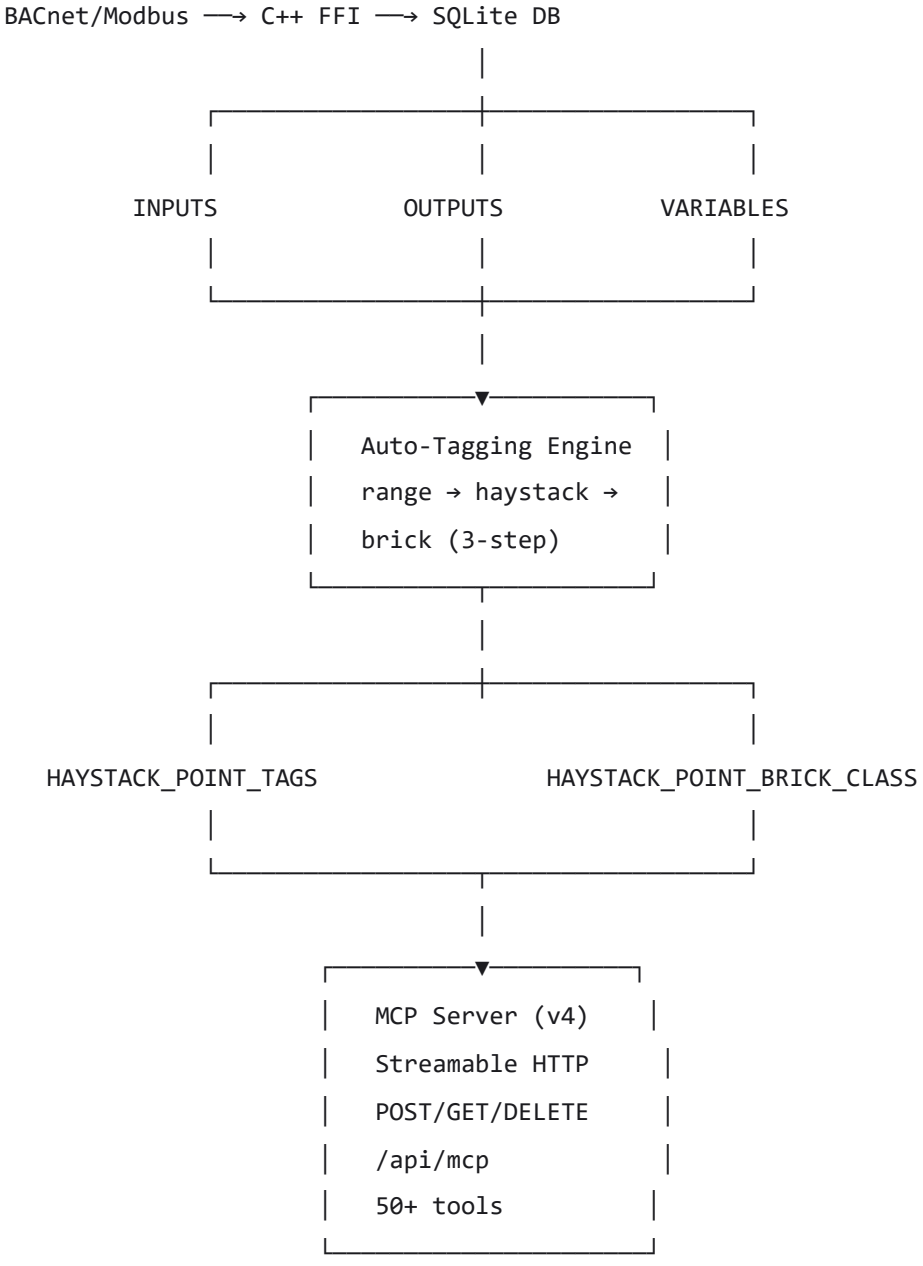
1. Scope

v4 extends the MCP server from 7 Haystack-only tools to 44 tools across 11 categories (since expanded to 50+ across 16 categories in v5). One file changes: `api/src/haystack/mcp.rs`. New handlers call existing public service functions — no other files touched.

	v3	v4
Tools	7	44
Categories	1 (haystack)	11
File changes	—	`mcp.rs` only

2. Architecture

TEXT



3. Tool Manifest

3.1 Haystack Tagging — 7 tools (existing, unchanged)

#	Tool	Input	Output
1	`t3000_haystack_list_tags`	`filter?: "haystack"|"brick"|"custom"`	`{ tags: [{ tag_name, category, doc, parents, source, usage_count }], total }`
2	`t3000_haystack_get_point_tags`	`serial_numbers: int[], point_type?: "INPUT"|"OUTPUT"|"VARIABLE"`	`[{ serial_number, point_type, point_index, point_id, tag_name }]`
3	`t3000_haystack_search_points`	`tags: string[], serial_numbers?: int[], point_types?: string[]`	`{ entries: [{ point_id, serial_number, point_type, point_index, tag_name }] }`
4	`t3000_haystack_auto_tag`	`serial_numbers: int[]`	`{ tagged_count: int }`
5	`t3000_haystack_preview_tags`	`serial_numbers: int[]`	`{ preview: [{ point_id, tags: [], brick_class }] }`
6	`t3000_haystack_list_rules`	—	`{ rules: [{ id, rule_name, category, pattern, haystack_tags, enabled }] }`
7	`t3000_haystack_get_brick_class`	`serial_numbers: int[]`	`[{ serial_number, point_type, point_index, brick_class }]`

3.2 Core — 3 tools (new)

#	Tool	Input	Output
8	`t3000_ping`	—	`{ status, timestamp, server }`
9	`t3000_get_version`	—	`{ name, version, protocolVersion, toolCount }`
10	`t3000_describe_tool`	`tool_name: string`	`{ name, description, inputSchema }`

3.3 Data & Metadata — 5 tools (new)

#	Tool	Input	Output
11	`t3000_device_list`	`filter_name?: string`, `refresh?: boolean`	`[{ serial, name, device_type, input_count, output_count, variable_count, online, last_checked, refreshed }]`
12	`t3000_device_get_points`	`serial_number: int`, `point_type?: "INPUT"|"OUTPUT"|"VARIABLE"`	`[{ point_type, point_index, label, engineering_units, haystack_tags, brick_class }]`
13	`t3000_point_get_metadata`	`serial_number: int`, `point_type: string`, `point_index: int`	`{ serial_number, point_type, point_index, label, engineering_units, range_low, range_high, description, haystack_tags, brick_class, digital_analog }`
14	`t3000_metadata_search`	`query: string`, `serial_numbers?: int[]`, `point_types?: string[]`, `limit?: int`	`[{ serial_number, point_type, point_index, label, haystack_tags, brick_class }]`
15	`t3000_point_search`	`query: string`, `serial_numbers?: int[]`, `point_types?: string[]`, `limit?: int`	`[{ serial_number, point_type, point_index, label, haystack_tags, brick_class }]`

3.4 Operational — 5 tools (new)

#	Tool	Input	Output
16	`t3000_point_read`	`serial_number: int`, `point_type: string`, `point_index: int`	`{ serial_number, point_type, point_index, label, value, engineering_units, timestamp }`
17	`t3000_point_write`	`serial_number: int`, `point_type: string`, `point_index: int`, `value: number|boolean`, `confirm: boolean`	`{ success, written_value, timestamp }`
18	`t3000_point_read_batch`	`points: [{ serial_number, point_type, point_index }]`	`[{ serial_number, point_type, point_index, label, value, engineering_units, timestamp }]`
19	`t3000_point_write_batch`	`points: [{ serial_number, point_type, point_index, value }]`, `confirm: boolean`	`{ success, count }`
20	`t3000_point_batch_metadata`	`points: [{ serial_number, point_type, point_index }]`	`[{ serial_number, point_type, point_index, label, engineering_units, haystack_tags, brick_class }]`

3.5 Analytics — 2 tools (new)

#	Tool	Input	Output
21	`t3000_haystack_validate`	`serial_numbers?: int[]`	`{ passed, warnings: [{ point_id, issue }], errors: [{ point_id, issue }] }`
22	`t3000_haystack_export`	`serial_numbers: int[]`, `format: "haystack-json"|"brick-ttl"|"brick-jsonld"`	Format-specific model dump

3.6 Rules Management — 2 tools (new)

#	Tool	Input	Output
23	`t3000_rule_toggle`	`rule_id: int`, `enabled: boolean`	`{ rule_id, rule_name, enabled }`
24	`t3000_rule_create`	`pattern: string`, `haystack_tags: string[]`, `category: "haystack"|"brick"`, `priority?: int`, `brick_class?: string`, `units?: string`, `object_types?: string`	`{ rule_id, rule_name, pattern, category, haystack_tags, enabled }`

3.7 Alarms & Trends — 3 tools (new)

#	Tool	Input	Output
25	`t3000_alarm_list`	`serial_numbers?: int[]`, `active_only?: boolean`	`[{ alarm_id, serial_number, name, severity, message, timestamp, acknowledged, acknowledged_at }]`
26	`t3000_alarm_acknowledge`	`serial_number: int`, `alarm_id: int`	`{ success, acknowledged_at }`
27	`t3000_trendlog_query`	`serial_number: int`, `point_type: string`, `point_index: int`, `start: ISO8601`, `end?: ISO8601`, `limit?: int`	`{ serial_number, point_type, point_index, label, engineering_units, data: [{ timestamp, value }] }`

3.8 Device Operations — 4 tools (new)

#	Tool	Input	Output
28	`t3000_trendlog_list`	`serial_number: int`	`[{ log_id, point_type, point_index, label, interval }]`
29	`t3000_trendlog_export`	`serial_number: int`, `format?: string`	`{ filename, data, format }`
30	`t3000_device_refresh`	`serial_number: int`	`{ success, message }`
31	`t3000_schedule_list`	`serial_number: int`	`[{ schedule_id, name, days, periods }]`

3.9 Settings — 3 tools (new)

#	Tool	Input	Output
32	`t3000_settings_read`	`serial_number: int`	`{ settings: { name, baud_rate, ... } }`
33	`t3000_settings_write`	`serial_number: int`, `settings: object`, `confirm: boolean`	`{ success }`
34	`t3000_device_control`	`serial_number: int`, `command: string`, `params?: object`	`{ success, message }`

3.10 Control Logic — 5 tools (new)

#	Tool	Input	Output
35	`t3000_program_list`	`serial_number: int`	`[{ program_id, name, size }]`
36	`t3000_program_read`	`serial_number: int`, `program_id: string`	`{ program_id, name, source }`
37	`t3000_alarm_settings_read`	`serial_number: int`	`[{ alarm_setting_id, point, condition, thresholds }]`
38	`t3000_users_list`	`serial_number: int`	`[{ user_id, name, access_level }]`
39	`t3000_graphics_list`	`serial_number: int`	`[{ graphic_id, label, picture_file }]`

3.11 Documentation — 5 tools (new)

#	Tool	Input	Output
40	`t3000_doc_list`	`section?: string`	`[{ path, title, section }]`
41	`t3000_doc_read`	`path: string`	`{ path, title, content }`
42	`t3000_pid_list`	`serial_number: int`	`[{ pid_id, name, setpoint, kp, ki, kd }]`
43	`t3000_holiday_list`	`serial_number: int`	`[{ holiday_id, name, start_date, end_date }]`
44	`t3000_building_summary`	`serial_numbers?: int[]`	`{ devices, points, tags, alarms }`

4. Naming Convention

Prefix	Scope	Count
(none)	Server-level	3
`haystack_`	Ontology: tags, Brick, auto-tagging, validation, export	9
`device_`	Device inventory, points, refresh, control	5
`point_`	Live values: read, write, batch, metadata	5
`rule_`	Auto-tagging rule CRUD	2
`metadata_`	Cross-point search	1
`alarm_`	Alarm query, acknowledge, settings	3
`trendlog_`	Historical data, listing, export	3
`schedule_`	Time-based schedules	1
`settings_`	Device configuration	2
`program_`	PLC program CRUD	2
`users_`	User management	1
`graphics_`	Visualization screens	1
`doc_`	Documentation articles	2
`pid_`	PID loop listing	1
`holiday_`	Holiday schedules	1
`building_`	System overview	1

5. Protocol

MCP Streamable HTTP (2025-03-26) over `/api/mcp`. No bridge needed.

5.1 Transport

Method	Endpoint	Purpose
`POST`	`/api/mcp`	JSON-RPC 2.0 requests (client → server)
`GET`	`/api/mcp`	SSE endpoint (server → client notifications)
`DELETE`	`/api/mcp`	Session termination

Session management via `Mcp-Session-Id` header — returned in every POST response, validated on GET/DELETE.

5.2 JSON-RPC Methods

Standard methods: `initialize`, `t3000_ping`, `tools/list`, `tools/call`. Notifications: `notifications/initialized` (accepted, no response).

```
TEXT
```

```
Request:
{ "jsonrpc": "2.0", "id": 1, "method": "tools/call",
  "params": { "name": "<tool>", "arguments": { ... } } }

Response (success):
{ "jsonrpc": "2.0", "id": 1,
  "result": { "content": [{ "type": "text", "text": "<json>" }] } }

Response (tool error — isError: true):
{ "jsonrpc": "2.0", "id": 1,
  "result": { "content": [{ "type": "text", "text": "<error>" }], "isError": true } }

Response (protocol error):
{ "jsonrpc": "2.0", "id": 1,
  "error": { "code": -32602, "message": "..." } }
```

5.3 MCP Compliance

Spec Requirement	Status
Streamable HTTP transport (POST + GET + DELETE)	✅
`Mcp-Session-Id` header	✅
`initialize` with `protocolVersion`, `serverInfo`, `capabilities`	✅ `2025-03-26`
`capabilities.tools.listChanged: true`	✅
`notifications/initialized` (no response)	✅
Protocol-level `t3000_ping`	✅
`tools/list` with `name`, `title`, `description`, `inputSchema`	✅
`tools/call` success: `{ content: [{ type: "text", text }] }`	✅
`tools/call` error: `{ content: [...], isError: true }`	✅
Standard JSON-RPC error codes	✅
Works with all modern MCP clients (VS Code, Claude, Cursor, etc.)	✅

6. Safety: Write Confirmation

`t3000_point_write` and `t3000_point_write_batch` reject writes to OUTPUT/VARIABLE points unless `confirm: true` is set. INPUT points are exempt.

`confirm`	OUTPUT/VARIABLE	INPUT
`false`	❌ Error	✅ Accepted
`true`	✅ Accepted	✅ Accepted

7. Validation Rules (`t3000_haystack_validate`)

Rule	Condition	Severity
`sensor` → INPUT	Point tagged `sensor` must be INPUT type	error
`cmd` → OUTPUT	Point tagged `cmd` must be OUTPUT type	error
`air` disambiguation	Tag `air` present without `temp` / `humidity` / `pressure` / `flow` / `quality`	warning
Brick type consistency	Brick class implies sensor → must be INPUT; implies actuator → must be OUTPUT	error
Orphaned tags	Tag in `HAYSTACK_POINT_TAGS` not in `HAYSTACK_TAGS`	warning

8. Export Formats (`t3000_haystack_export`)

Format	MIME	Structure
`haystack-json`	`application/json`	`[{ id, dis, tags: { ... } }]` — Project Haystack v4
`brick-ttl`	`text/turtle`	`@prefix brick: <https://brickschema.org/schema/Brick#> .` — OWL/RDF
`brick-jsonld`	`application/ld+json`	`{ "@context": {...}, "@graph": [...] }` — JSON-LD

9. Implementation

9.1 Module: `api/src/haystack/mcp.rs`

Section	Change
`TOOLS` lazy_static	All 44 `ToolDef` entries with `name`, `title`, `description`, `inputSchema`
`handle_request` match	4 methods: `initialize`, `t3000_ping`, `tools/list`, `tools/call`
`mcp_post_handler`	Session management via `Mcp-Session-Id`, notification handling
`mcp_sse_handler`	GET /api/mcp SSE endpoint
`mcp_delete_handler`	DELETE /api/mcp session termination
`handle_tools_call`	`isError: true` for tool errors (LLM-friendly)
`execute_tool`	44 tool handlers + input validation

9.2 Service Dependencies

Tool	Calls
`t3000_ping`, `t3000_get_version`, `t3000_describe_tool`	Static / `TOOLS` array
`t3000_device_list`	`T3DeviceService`
`t3000_device_get_points`, `t3000_point_get_metadata`	Point tables + `tags_service` + Brick class query
`t3000_metadata_search`	`tags_service::search_points`
`t3000_point_read`, `t3000_point_read_batch`	Raw SQL SELECT fValue
`t3000_point_write`, `t3000_point_write_batch`	Raw SQL UPDATE fValue
`t3000_haystack_validate`	`HAYSTACK_POINT_TAGS` + `HAYSTACK_POINT_BRICK_CLASS` + point tables
`t3000_haystack_export`	All tagged points → serialize
`t3000_rule_toggle`, `t3000_rule_create`	`auto_tagging_service`
`t3000_alarm_list`, `t3000_alarm_acknowledge`	Alarm tables
`t3000_trendlog_query`	`T3TrendlogDataService`

9.3 Files Not Touched (unchanged from v4.0)

- `api/src/server.rs` — no route changes (MCP already registered)
- `api/src/t3_device/` — no changes (existing services used as-is)
- `api/src/haystack/tags_routes.rs` — no changes
- `api/src/haystack/auto_tagging_routes.rs` — no changes
- `api/src/haystack/tags_service.rs` — no changes
- `api/src/haystack/auto_tagging_service.rs` — no changes
- All frontend files — no changes
- All database migrations — no changes

9.4 Removed (v4.1)

- `tools/t3000-mcp-bridge.js` — no longer needed; server speaks Streamable HTTP natively

---

10. Changelog

v4.1 — 2026-07-20 (MCP Streamable HTTP compliance)

Change	Detail
Transport	POST + GET(SSE) + DELETE `/api/mcp` (Streamable HTTP)
Sessions	UUID sessions via `Mcp-Session-Id` header
Protocol version	`2025-03-26`
Capabilities	`tools.listChanged: true`
`notifications/initialized`	Accepted as JSON-RPC notification (no response)
Protocol `t3000_ping`	Added separate from tool `t3000_ping`
Tool errors	`isError: true` in result (not JSON-RPC error)
Tool `title`	All 44 tools have `title` field
`t3000_describe_tool`	Includes `title` in output
`t3000_get_version`	Uses `PROTOCOL_VERSION` constant
`t3000_point_get_metadata`	`point_type` validated before SQL (security)
Bridge	Removed `tools/t3000-mcp-bridge.js` — no longer needed

v4.0 — 2026-07-20

Change	Detail
Tools	Extended from 7 to 44 across 11 categories
Categories	Haystack, Core, Data, Operational, Analytics, Rules, Alarms
File changes	`mcp.rs` only

11. Database Tables (unchanged)

Table	Rows (approx)
`HAYSTACK_TAGS`	719
`HAYSTACK_TAG_RELATIONS`	~800
`HAYSTACK_POINT_TAGS`	533
`HAYSTACK_POINT_BRICK_CLASS`	~533
`HAYSTACK_AUTO_TAGGING_RULES`	228
`INPUTS` / `OUTPUTS` / `VARIABLES`	~192 per device
`ALARMS`	device-dependent
`TRENDLOG_DATA`	device-dependent

Version 4.0 — 2026-07-20